

" Everything in Javascript happens inside a Execution Context "

→ It has two components.

### 1. Memory Component

This is the place where all variables and functions are stored as a key value pairs.

• Memory component also known as Variable Environment.

### 2. Code component

This is the place where code is executed one line at a time.

• This is also known as Thread of Execution.

| Memory     | code |
|------------|------|
| Key: value | • {  |
| a: 10      | • }  |
| fn: f... 3 | • }  |

• Two phases  
 → Memory creation  
 → Code execution phase

Note:

" Javascript is a synchronous single threaded language. "

→ It means javascript can <sup>only</sup> execute one command at a time

synchronous single thread means & JS can only execute one command at a time and in a specific order.

undefined is a keyword.

## \* Function invocation :

Whenever we see a function name with the parenthesis, it means now the function is executed.

Functions are the heart of JavaScript. In JS, functions behave very differently than any other language.

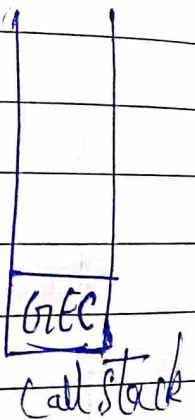
## \* Callstack

Callstack handles everything to manage the execution context creation, deletion, and the control, it manages the stack which is also known as callstack.

Callstack is a stack

• Everytime in the bottom of the stack we have our global execution context

"Callstack maintains the order of execution of execution contexts"



→ Callstack also known as

1. Execution Context Stack
2. Program Stack
3. Control Stack
4. Runtime Stack
5. Machine Stack

## \* Hoisting in Javascript

Hoisting is a phenomenon in JS by which we can access variables and functions even before initialisation or put have some value in it without any error.

\* "Even before the code starts executing, memory is allocated to all variables and functions"

⇒ When the variable (let's say) is not present everywhere in the memory or not initialised anywhere in the program. Then it throws an error, which is "not-defined"

## \* Undefined:

It is like a place holder which is placed inside a memory.

- It takes the memory.

It is a special keyword. It takes up own memory like a place holder which is kept for time being until the variable is assigned for some other value.

- JS is a loosely typed language.

## \* Scope:

means where we can access a specific variable or a function in our code.

Scope is directly depend on the lexical environment.

## \* Lexical environment

"Whenever an execution context is created a lexical environment is also created."

- lexical env. is the local memory along with <sup>the</sup> lexical env. of its parent.
- LE is the local memory and reference to lexical env. of its parent.

## \* let & const

"let & const declarations are Hoisted"  
 "These are in the temporal deadzone for the time being."

temporal deadzone is the time since when let variable was hoisted and till it is utilized some value the time b/w that is known as TD

When we tried to access a variable in the temporal deadzone, it gives a Reference Error

## o Window

Window is actually a global object which is created along with the global execution context.

⇒ So whenever a JavaScript program is run, a global object is created, a global execution context is created and along with execution context a `this` variable is created.

`this === window`

⇒ Global space is a space where we write any code in JS which is not inside a function.

Anything which is not inside a function that is global space.

### \* Block scope

Block is also known as Compound statement.

- Block is used to combine multiple JavaScript statements into one group.

⇒ We group multiple statements together in a block so that we can use it where JavaScript expects one statement. ⇐

⇒ Block scope is what all variables and functions we can access inside this block.

⇒ `var` is a global scope.

`let` & `const` are block scope that means they are stored in a separate memory space which is reserved for this block.

## \* shadowing

## \* Closures

"A function binds together with its lexical environment"

Function along with its lexical scope forms a closure.

```
function x() {
```

```
  var age = 7;
```

```
  function y() {
```

```
    console.log(a);
```

```
  }
```

```
  return y;
```

```
}
```

```
var z = x();
```

```
console.log(z);
```

```
z();
```

A closure is a function that remembers its outer variables and can access them.

• All functions in javascript are closures.

- Function bundled with its lexical environment is known as closure. Whenever function is returned, even if it is vanished in execution context but it still remembers the environment it was pointing to. It's not just that function along with its return But the entire closure and that's where it becomes interesting.

## > Uses of Closures

- Module Design Pattern
- Currying
- Function like once
- Memoize
- maintaining state in async world
- setTimeouts
- Iterators
- and many more ..

②

```
function outer(b) {
```

```
  function inner() {
    console.log(a, b);
  }
```

```
  let a = 10;
  return inner
```

```
}
var close = outer("Hello");
close();
```

Inner function forms a closure with its outer environment and  $b$  is also part of a outer environment of inner function, so it forms closure with  $b$  also.

⇒ closures helps in data hiding and encapsulation

## → Disadvantages of Closures

### 1. <sup>Over</sup> Memory Consumption:

Because every time closures are found, so it consumes lot of memory and sometimes ~~on~~ these closed variables are not garbage collected. So that means it is kind of accumulating lot of memory if we create lot of closures in our programs. Because those variables are not garbage collected till the program expires.

2) It can also lead to memory leaks.

## \* Garbage Collector:

Garbage collector is a program in the browser or the JS Engine which kind of freeze unutilised memory.

## \* Functions

### → Function statement

```
function a() {  
  console.log("Hello");  
}
```

The function is normally defined and can be used later on.

By this we can do Hoisting

### → Function Expression

```
b(); // error  
var b = function () {  
  console.log("World");  
}
```

## → Disadvantages of Closures

### 1. <sup>over</sup> Memory Consumption:

Because every time closures are found, so it consumes lot of memory, and sometimes ~~on~~ these closed variables are not garbage collected. So that means it is kind of accumulating lot of memory if we create lot of closures in our programs. Because those variables are not garbage collected till the program expires.

2) It can also lead to memory leaks.

## \* Garbage Collector:

Garbage collector is a program in the browser or the JS Engine which kind of freeze unutilised memory.

## \* Functions

### → Function statement

```
function a() {  
  console.log("Hello");  
}
```

The function is normally defined and can be used later on.

By this we can do Hoisting

### → Function Expression

```
b(); // error  
var b = function() {  
  console.log("World");  
}
```

function expression simply means to create a function and put it into the variable.

//// Dff b/w function statement and function Expression.

A function statement loads before any any code is executed. This behavior of function statement is called hoisting, which allows a function to be used before it is defined.

A function expression associates a value with a variable, just like any other assignment statement.

⊗ Function statement also known as function declaration

→ Anonymous function

Function without a name is known as Anonymous function.

• Anonymous function does not have their identity.

function () {

}

// This will show error  
Cuz require name

Used: Anonymous function are used in a place where functions are used as value.

→ Named function:  
A normal function with its name assigned to a variable.

// In this case we can't call function by its name in outer scope!

```
var b = function xyz() {  
    console.log("Hi Sachin");  
}
```

xyz → not error

xyz(); // It will throw error

→ Parameters & Arguments

```
var b = function (para1, para2) {  
    console.log("Unknown");  
}
```

parameters

b(1, 2);

Arguments

→ first class function or First Class citizens

The ability of use function as value

- Can be used as an Argument
- Can be executed inside a closure function &
- Can be taken as return form.

```
var b = function (param) {  
    // return function xyz() {  
        console.log("Fct");  
    }  
}
```

Functions are heart of JS. They are called first class citizens or first class functions because they have the ability to be stored in the variables, passed as parameters and arguments. They can also be returned in the function.

### \* What is callback function

function that is passed on as argument to another function is called callback function. It gives us access to the whole asyn. world in a syn. single threaded language.

- setTimeout helps turns JS which is single threaded and synchronous into asynchronous.
- Event listeners can also invoke closures with scope.
- Event listeners consumes a lot of memory which can potentially slowdown the websites therefore it is good practice to remove it if it is not used.

### \* Event Loop

JavaScript is a single synchronous threaded language. It has one call stack and it can do only one thing at a time.

This callstack is present inside javascript engine and all the codes are executed in the callstack.

⇒ set Timeout is not a part of JavaScript  
DOM APIs - local storage  
console  
Location  
fetch()

→ To keep checking whether the callstack is empty or not

## \* Event loop

Event loop is an endless loop, which waits for tasks, executes them and then sleeps until it receives more tasks.

The event loop executes tasks from the event queue only when the callstack is empty.

The event loop allow us to use callbacks and promises.

## → Callback queue :

Task Queue

callback queue gets the ordinary callback functions coming from the set Timeout() API after the timer expires.

→ Fetch : basically goes and requests a API call.

## → Microtask Queue

All the callback functions which comes through Promises<sup>and Mutation Observers</sup> would go inside the microtask queue.

callback queue has lesser priority than Microtask queue.

Mutation Observer interface provides the ability to watch for changes being made to the DOM tree.

## \* Starvation :

Starvation is usually caused by an overly simplistic scheduling algorithm.

Ex. If a multi-tasking system always switches between the first two tasks while a third never gets to run, then the third task is being starved of CPU time.

## \* Asynchronous :

In Asy. operations we can move to another task before the previous one finishes,

⇒ First JS Engine ⇒ SpiderMonkey ⇒ Firefox

⇒ JS Engine is not a machine.

It is just like a normal program like a normal code which is written in low level language.

For example ⇒ Google V8 engine is written in C++

Interpreter

Compiler

JIT Compilation

It uses both interpreter and compiler to execute the code

Tokens are generated

code

JS Engine

Page No.

Date

Parsing

JIT Compilation

AST

Interpreter

Compilation

Compiler

Execution

GC

Memory heap

It tries to free memory space whenever as possible

callstack

Forms of optimisation which compiler does for the code like

Inlining, Inlining caching, copy elision

This is the place where all the fun & where all the memory variables are assigned is store of memory

⇒ Garbage Collector ⇒ whenever no one function is not used or we clear the timeout so basically it collects the all garbage and sweeps it. It uses Mark & Sweep Algorithm



V8 ⇒ has a interpreter known as Ignition They have Turbo fan optimising compiler which does the job of compilation very fast

Avanoco ⇒ Garbage Collector  
⇒ uses mark & sweep algorithm

JS is JIT compiled language.

- ⑥ Mark and Sweep algorithm is called a tracing garbage collector because it traces out the entire collection of objects that are directly or indirectly accessible by the program.

### Mark & Sweep

#### Mark Phase

when obj. is created its mark bit set to 0 (false)  
for reachable obj. we set mark bit to 1 (true).

#### Sweep Phase

It sweeps the unreachable obj. i.e. it clears the heap memory for all the unreachable obj.

### \* Higher Order functions :

A function which takes another function as an argument or returns a function from it is known as high order function.

### \* Prototype

Whenever we create any object, JS engine automatically puts this hidden property into an object and attaches an object to the original object.

- The value of proto can be either an object or null.

\* Class  
In JS, class is a type of functions

\* Callbacks

1. Issues with callbacks

- a. callback hell
- b. Inversion of control

a. Callback hell: Asynchronous operations in JS. can be achieved through callbacks. Whenever there are multiple dependent Asynchronous operations it will result in a lot of nested callbacks. This will cause a 'pyramid of doom' like structure.

b. Inversion of control: When we give the control of callbacks being called to some other API, this may create a lot of issues. That API may be buggy, written by interns, may not call our callback and create order as in the example may call the payment call twice.

\* Promises

Javascript is a single threaded, meaning that two bits of script cannot run at the same time. They have to run one after another.

A promise is an object that represents the eventual completion (or failure) of an asynchronous operation, and its resulting value.

## \* Async / Await

Async make a function return a promise.  
await make a function wait for a promise.

- ⇒ the `await` keyword can only be used inside an `async` function.
- ⇒ `async` keyword can be placed before a function like this.

- So, `async` ensures that the function returns a promise and wraps non-promises in it.

- The keyword `await` makes Javascript wait until that promises settles and returns its result.

\* API (Application Programming Interface)

AJAX (Asynchronous Javascript And XML)

JSON (Javascript Object Notation)

↳ text format for storing and transporting data.

• Commonly used for API and Config files

→ Why use JSON? JSON is language independent.

⇒ A javascript program can easily convert JSON data into Javascript objects.

- A common use of JSON is to exchange data to/from a web server.
- When receiving data from a web server, the data is always a <sup>string</sup>.
- Parse the data with `JSON.parse()`, and the data becomes a Javascript object.